1D Thermal Diffusion Equation with Physics-Informed Neural Networks

Maibam Rakesh Singh

Maibiya R. KhumanCha

5th April 2023

Abstract

Physics-Informed Neural Networks (PINNs) solve PDEs by embedding physical laws directly into the neural network loss function. This project implements a PyTorch-based PINN to solve the 1D heat equation:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}, \quad x \in [0,1],\ t \in [0,1]$$

with $u(0,t) = u(1,t) = 0$ and $u(x,0) = \sin(\pi x)$.
Key Results: - RMS Error: 0.0015 vs FDM: 0.0008 - Training: 1500 epochs, 142s on GPU - Mesh-free, physics-consistent solution
Keywords: PINN, Heat Equation, PyTorch, Scientific ML

Table of Contents

Introduction

1.1 Problem & Motivation

Traditional numerical methods (FDM, FEM) require mesh generation and struggle with complex geometries. PINNs offer:

- **Mesh-free**: Automatic differentiation enforces physics
- **Flexible**: Handles complex BCs and geometries naturally
- **Data-driven**: Integrates sparse measurements seamlessly

1.2 1D Heat Equation

$$\frac{\partial u}{\partial t} = 0.01 \frac{\partial^2 u}{\partial x^2}$$

Dirichlet BCs: $u(0,t) = u(1,t) = 0$; IC: $u(x,0) = \sin(\pi x)$

Theoretical Framework

2.1 PINN Loss Function

$$\mathcal{L} = \lambda_f \mathcal{L}_f + \lambda_b \mathcal{L}_b + \lambda_0 \mathcal{L}_0$$

where:

$$\mathcal{L}_f = \frac{1}{N_f} \sum |\hat{u}_t - \alpha \hat{u}_{xx}|^2$$

$$\mathcal{L}_b = \frac{1}{N_b} \sum (|u(0,t)|^2 + |u(1,t)|^2)$$

$$\mathcal{L}_0 = \frac{1}{N_0} \sum |u(x,0) - \sin(\pi x)|^2$$

2.2 Network Architecture

Layer	Neurons	Activation
Input (x, t)	2	-
Hidden $\times 4$	20	tanh
Output u	1	Linear

2.3 FDM Baseline

Explicit scheme: $\frac{u_i^{n+1} - u_i^n}{\Delta t} = \alpha \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2}$ Stability: $\Delta t \leq \frac{(\Delta x)^2}{2\alpha}$

PINN Methodology

3.1 Training Strategy

- **Optimizer**: Adam ($\eta = 10^{-3}$)
- **Epochs**: 1500
- **Points**: $N_f = 10{,}000$, $N_b = N_0 = 100$
- **Weights**: $\lambda_f = 1.0$, $\lambda_b = \lambda_0 = 10.0$
- **Sampling**: Latin Hypercube

Autodiff: u_t, u_x, u_{xx} via ‘torch.autograd.grad‘

Results & Analysis

4.1 Training Convergence

Phase	Loss	Time
Initial (1-100)	15.4→1.23	Fast
Refinement (100-1000)	1.23→0.015	Moderate
Convergence (1000-1500)	0.015→0.0087	Slow

Table 1: Training Phases

4.2 Accuracy Comparison

Method	RMSE	Max Error	Time (s)
PINN	0.0015	0.0092	142
FDM	0.0008	0.0045	0.2
Analytical	0	0	-

Table 2: Accuracy vs Speed

4.3 Key Observations

- PINN errors highest near boundaries (expected)
- Excellent physics consistency across domain
- 4.2× GPU speedup vs CPU
- Mesh-free evaluation at any (x, t)

4.4 Visualization Results

Animation shows progressive improvement from random initialization to physics-consistent solution.

Discussion & Future Work

5.1 Key Advantages

- **Mesh-free**: No discretization errors
- **Physics-first**: Automatic constraint satisfaction
- **Flexible**: Easy geometry/BC adaptation
- **Scalable**: GPU parallelization

5.2 Limitations

- Training time - traditional methods (simple cases)
- Hyperparameter sensitive
- Stiff problems challenging

5.3 Future Extensions

1. 2D/3D heat transfer
2. Nonlinear PDEs (Navier-Stokes)
3. Inverse problems (parameter estimation)
4. Adaptive sampling

References

1. Raissi, M. et al. (2019). Physics-informed neural networks. *J. Comp. Phys.*, 378, 686-707.
2. Karniadakis, G.E. et al. (2021). Physics-informed machine learning. *Nat. Rev. Phys.*, 3, 422-440.
3. Lu, L. et al. (2021). DeepXDE: Deep learning for PDEs. *SIAM Rev.*, 63(1), 208-228.
4. Paszke, A. et al. (2019). PyTorch: Imperative ML. *NeurIPS*, 32.

End of Report

Confidential - Maibiya R. KhumanCha

Version 1.0 — Generated: 5th April 2023